

FrontierAtlas Intelligence Ingestion

Production architecture for source-traceable AI ecosystem data | August 2026

Objective

Collect startups, products, research papers, jobs, and news as verifiable graph-ready records. The design prioritizes provenance and repeatability: no record may be emitted without a legitimate source URL; language models may structure evidence but may not invent facts.

1. Scale strategy - 500,000+ entity records

Layer	Design	Scale mechanism
Discovery	Scheduled API/RSS/directory enumerators emit immutable URL discovery events.	Partition by source + cursor/range; queue absorbs burst traffic.
Fetch	Stateless async workers enforce global and per-domain token buckets; raw HTML/PDF goes to object storage.	Autoscale workers on queue lag; use source-specific concurrency caps.
Transform	Schema validators, deterministic parsers, GitHub metric enrichers, and LLM extraction jobs consume raw snapshots.	Idempotency key = normalized URL + content hash + schema version.
Serve	Postgres holds canonical entities/provenance; graph/vector indexes are derived views.	Bulk upsert and append-only event history avoid cross-worker conflicts.

A large one-time backfill is a cursor-driven workflow, not a single long process: enumerators checkpoint source pages, each URL is placed on a durable queue, and workers write terminal outcomes. Failed tasks use a dead-letter queue with reason and retry schedule. Capacity changes (workers, queue partitions, and database IOPS) increase throughput without code changes.

2. Record quality and entity resolution

Every schema includes source name, original URL, collection timestamp, and a schema version. Startup/product imports reject missing or malformed URLs. Research paper records originate from arXiv; GitHub star counts are retrieved only from the GitHub API and timestamped on collection. The resolver first applies a conservative normalization (case, punctuation, corporate suffixes), then thresholded fuzzy matching against seed canonicals. Low-confidence matches remain unresolved and are logged rather than silently merged.

Operational invariant: preserve raw snapshot, extracted field evidence, resolver version, and source URL so any record can be audited or reprocessed.

Reliability and Freshness Controls

Failure-tolerant extraction at high concurrency

3. Exact handling of 413 and 429

Condition	Worker behavior	Safety property
413 Payload Too Large	Never submit whole raw pages. First retain the raw snapshot, extract readable article text, then chunk on paragraph/sentence boundaries under a provider-specific character/token budget. Retry only the smaller chunk.	Dense content is retained while request size is bounded; chunk identifiers retain parent URL and offsets.
429 Rate Limited	Honor Retry-After when supplied; otherwise exponential backoff with jitter. Reduce the affected provider/source concurrency and proceed with the next configured LLM provider when appropriate.	Avoids synchronized retry storms and protects independent sources from one provider outage.
Timeout / 5xx	Bounded retries with jitter; after exhaustion persist a retryable failure to the queue/DLQ.	No task disappears and no request loops indefinitely.

LLM chain

The production order is Gemini Flash, Groq-hosted Llama, then DeepSeek, subject to configured credentials and policy. Providers expose a common extract interface. Prompt contracts require JSON and explicitly forbid unsupported inference. Validate JSON against the target schema, retain per-field source snippets, and reject records without provenance. Deterministic parsers handle dates and common metadata before any LLM call to reduce cost and hallucination risk.

4. Freshness and distributed deduplication

For news and jobs, the worker normalizes RSS/Atom timestamps and page metadata to UTC. A record is accepted only if published in the last 24 hours (with a small future-skew tolerance). Relative labels such as “2 hours ago” are evaluated against collection time. When no trustworthy date exists, do not claim freshness: store the URL as pending and use a source-specific change detector (first-seen time, feed position, content hash, and next-run confirmation) for review.

A globally unique ledger keyed by SHA-256(normalized URL) prevents multiple nodes from accepting the same URL. A second key, URL + content hash + parser/schema version, controls reprocessing when pages change. In Postgres, `INSERT ... ON CONFLICT DO NOTHING` claims a URL atomically; event rows record fetch, parse, quality-gate, and export outcomes. This provides exactly-once acceptance even though fetching remains at-least-once.

5. Monitoring

Emit structured logs with run ID, source, URL hash, status, retry count, latency, freshness decision, and resolver confidence. Alert on queue lag, 429/403 spikes, stale feeds, parser errors, LLM fallback frequency, and abnormal source volume. Dashboards separate availability from verified data quality.

Storage, Compliance, and Delivery

Graph-ready data without sacrificing raw evidence

6. Storage strategy

Data	System	Reason
Raw responses	S3/GCS-compatible object storage	Low-cost immutable source snapshots; retention and replay.
Canonical records + ledger	Postgres	ACID URL claims, relational provenance, JSONB evolving schemas, straightforward bulk upserts.
Relationships	Neo4j or Postgres/Apache AGE derived graph	Maps startup-founder-product-paper-job relationships and supports graph traversal.
Semantic retrieval	pgvector initially; dedicated vector service if scale warrants	Keeps embeddings near metadata; migrate only when latency/volume requires it.
Analytics	Parquet in a lakehouse + warehouse	Cheap historical scans and reproducible model features.

7. Anti-bot and JavaScript-rendered sources

The appropriate strategy is compliance-first, not CAPTCHA bypass. Prefer licensed data, documented APIs, RSS feeds, partner exports, or explicit written permission. For a permitted JavaScript-rendered source, run Playwright asynchronously with a persistent, rate-limited browser context, render only pages discovered from allowed indexes, and cache snapshots. Observe robots.txt, terms, source-specific request budgets, and a kill switch. For Cloudflare/DataDome blocks, stop automated requests, record the block signal, and escalate to an official API, commercial data agreement, or manual authorization workflow. The pipeline must never attempt to defeat CAPTCHAs or access controls.

8. Deployment and acceptance

Deploy containers as scheduled discovery jobs plus queue consumers. Secrets live in a managed vault; source credentials and LLM keys never enter logs or exports. CI runs unit tests, schema fixtures, date-parsing regressions, and mocked 413/429 retries. A staging run validates row-level provenance before production promotion. The export service writes six tabs/files: Startups, Products, Research Papers, Jobs, News, and Entity Mapping Log.

Acceptance checklist

- 1,000 distinct source-backed startups and products supplied by approved directory/API imports.
- 1,000 arXiv/Papers with Code research papers, with GitHub URL/star values only when a source association is verified.
- All exported jobs/news pass the 24-hour UTC gate; undated items are excluded or marked pending.
- Every exported entity maps to its original source URL; resolver decisions and operational failures are auditable.

Implementation companion: ``README.md`` and ``src/intel_pipeline/``